DeepLeaf documentation

PageIndex is a tool that reads a PDF and builds a structured map of it. Instead of just extracting all the text page by page, it tries to understand the document's structure:

```
PDF
└── Section
    └── Subsection
        └── Page range
```

The output is usually a JSON tree showing:

- section titles
- nested subsections
- start pages (start index)
- end pages (end index)
- optional node IDs
- optional node summaries (generated via LLM)
- optional document description
- optional parent-child structure between sections

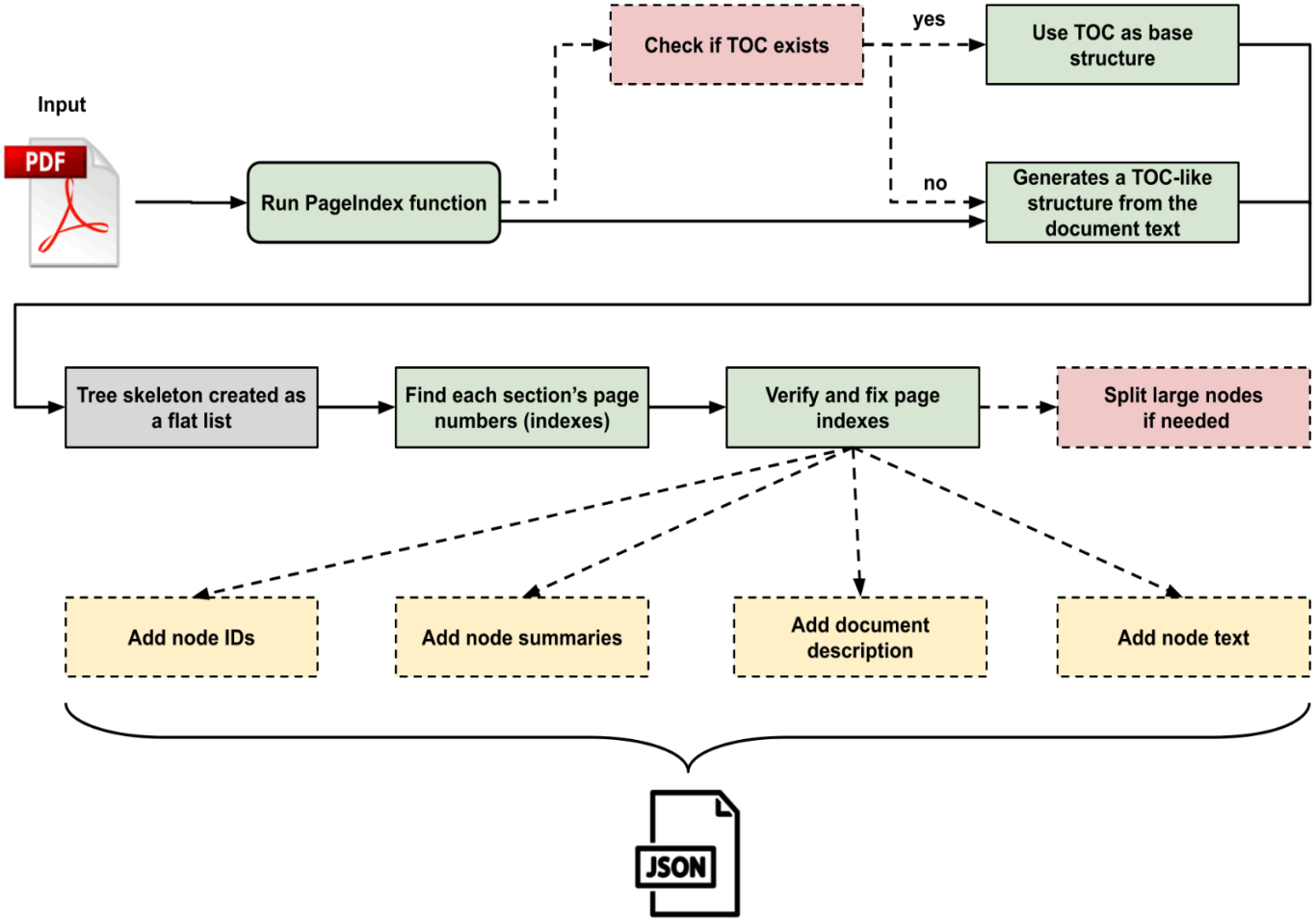
PageIndex settings can be modified in the YAML configuration file (./pageindex/config.yaml).

Example YAML parameters:

```
toc_check_page_num: 5
max_page_num_each_node: 5
max_token_num_each_node: 5000
if_add_node_id: "yes"
if_add_node_summary: "yes"
if_add_doc_description: "no"
if_add_node_text: "no"
```

These parameters control how PageIndex checks for a table of contents, how large each node can be, and what extra information gets added to the final JSON output.

PageIndex pipeline



Detailed explanation of PageIndex pipeline:

1. Input PDF
   PageIndex receives the PDF file that needs to be indexed.
   LLM used: No.

2. Extract page-level text
   PageIndex extracts text from the PDF while keeping the text linked to its page number.
   It also calculates token length for each page.
   This extracted page list is reused throughout the rest of the pipeline.
   LLM used: No.
   Code detail: get_page_tokens() uses PyPDF2 or PyMuPDF to extract page text and token counts. :contentReference[oaicite:0]{index=0}

3. Check for a table of contents
   PageIndex uses the extracted page text to check the first few pages for a TOC.
   This is controlled by:
   toc_check_page_num: 5
   LLM used: Yes.
   Code detail: find_toc_pages() calls toc_detector_single_page(), and toc_detector_single_page() sends the page text to the LLM to decide whether that page contains a table of contents. :contentReference[oaicite:1]{index=1}

4. Use or generate document structure
   If a usable TOC exists, PageIndex uses it as the base structure.
   If no usable TOC exists, PageIndex generates a TOC-like structure from the document text.
   LLM used: Yes.
   Code detail:
   - If a TOC exists, toc_transformer() uses the LLM to convert the raw TOC into JSON.
   - If no TOC exists, process_no_toc() calls generate_toc_init() and generate_toc_continue(), which use the LLM to generate a hierarchical structure from the document text. :contentReference[oaicite:2]{index=2}

5. Detect section page indexes
   PageIndex uses the extracted page text to find where each section starts and ends.
   These become:
   - start_index
   - end_index
   LLM used: Yes for finding physical_index / section starts.
   Code detail:
   - toc_index_extractor() uses the LLM to add physical_index values to TOC items.
   - add_page_number_to_toc() uses the LLM to check where section titles start in tagged page text.
   - Later, post_processing() converts physical_index into start_index and end_index. :contentReference[oaicite:3]{index=3}

6. Verify and fix page indexes

PageIndex checks whether the detected indexes are valid and tries to correct them if needed.

LLM used: Yes.

Code detail:

- verify_toc() calls check_title_appearance(), which uses the LLM to check whether a section title actually appears/starts on the detected page.
- If indexes are wrong, fix_incorrect_toc_with_retries() can call LLM-based fixing logic.
- validate_and_truncate_physical_indices() itself is not LLM-based; it only removes impossible page indexes that exceed the PDF length. :contentReference[oaicite:4]{index=4}

7. Split large nodes if needed

Optional / configurable.

PageIndex can limit how large each node is using:

max_page_num_each_node: 5

max_token_num_each_node: 5000

LLM used: Yes, but only if a node is too large.

Code detail: process_large_node_recursively() checks whether a node exceeds the page/token limits. If it does, it reruns meta_processor() in process_no_toc mode on that node's page range, which uses the LLM to create a smaller internal structure. :contentReference[oaicite:5]{index=5}

8. Build the JSON tree

PageIndex converts the document structure into a nested tree:

```
PDF
└── Section
    └── Subsection
        └── Page range
```

LLM used: No for the actual tree conversion.

Code detail: post_processing() sets start_index/end_index, then list_to_tree() builds the nested parent-child tree from the flat structure list. :contentReference[oaicite:6]{index=6}

9. Add node IDs

Optional.

Controlled by:

if_add_node_id: "yes"

LLM used: No.

Code detail: write_node_id() just assigns numbered IDs like 0000, 0001, etc. :contentReference[oaicite:7]{index=7}

10. Add node summaries

Optional.

Controlled by:

if_add_node_summary: "yes"

LLM used: Yes.

Code detail: generate_summaries_for_structure() calls generate_node_summary() for each node, and generate_node_summary() sends the node text to the LLM to create a summary. :contentReference[oaicite:8]{index=8}

11. Add document description
Optional.
Controlled by:
if_add_doc_description: "no"
LLM used: Yes, only if enabled.
Code detail: generate_doc_description() sends the cleaned document structure to the LLM and asks for a one-sentence document description. :contentReference[oaicite:9]{index=9}

12. Add node text
Optional.
Controlled by:
if_add_node_text: "no"
LLM used: No.
Code detail: add_node_text() copies the already-extracted page text into each node based on start_index and end_index. :contentReference[oaicite:10]{index=10}

13. Output structured JSON
The final output is a JSON tree containing the PDF structure, page ranges, and any optional metadata enabled in the YAML file.
LLM used: No for saving/final formatting.
Code detail: page_index_main() returns a dictionary with doc_name and structure, and optionally doc_description if enabled. :contentReference[oaicite:11]{index=11}

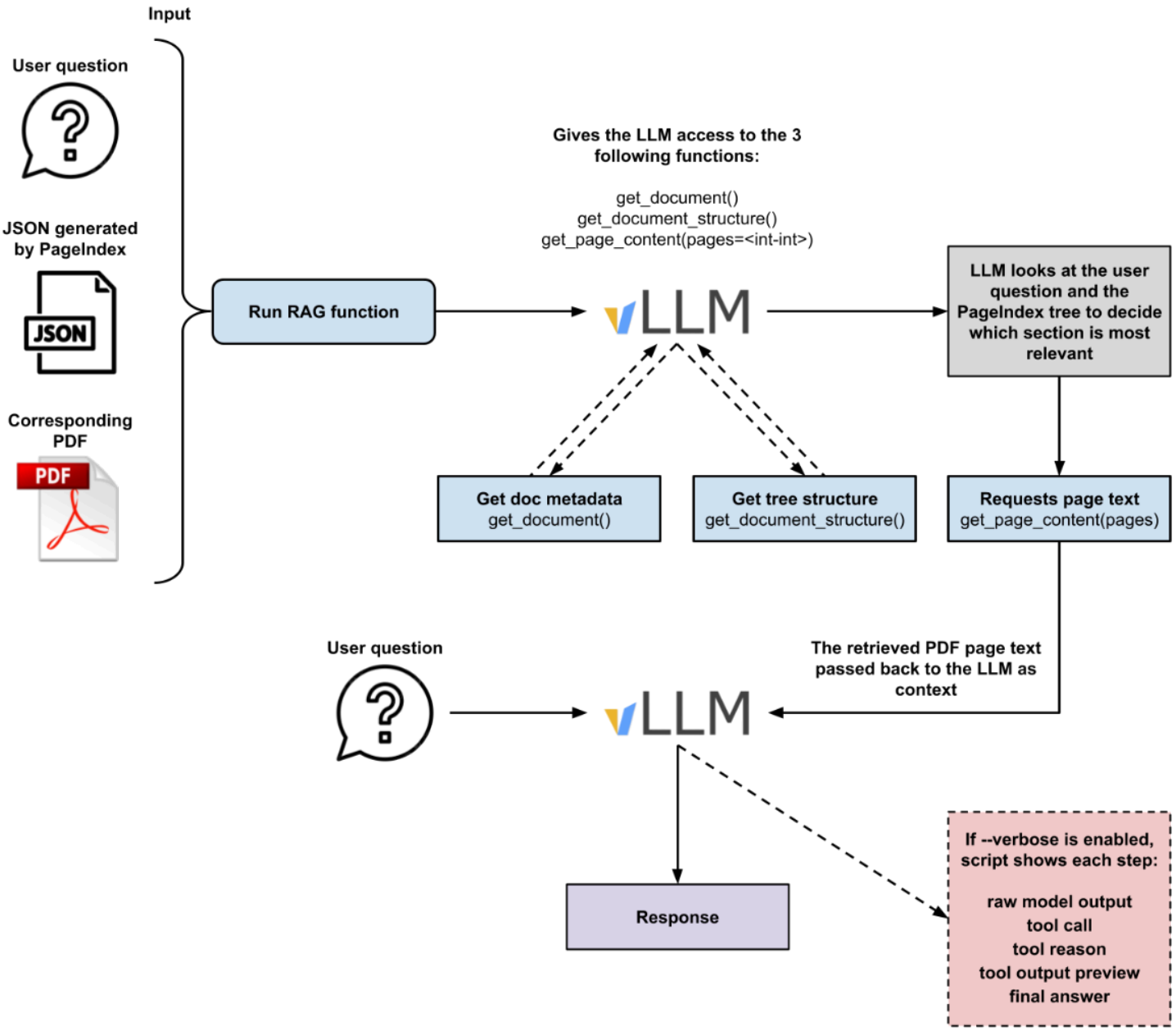
Example after step 4/5:

```
[
 {
   "structure": "1",
   "title": "Introduction",
   "physical_index": 1
 },
 {
   "structure": "1.1",
   "title": "Background",
   "physical_index": 2
 }
]
```

Then in step 8, PageIndex turns that skeleton into the actual nested tree:

```
[
 {
   "title": "Introduction",
   "start_index": 1,
   "end_index": 1,
   "nodes": [
     {
       "title": "Background",
       "start_index": 2,
       "end_index": 2
     }
   ]
 }
]
```

PageIndex RAG pipeline



Detailed explanation of PageIndex pipeline:

1. User question
   The user asks a question about the PDF.

2. Load the PDF
   The RAG script receives the PDF path and loads the PDF page by page.
   Example:

   docs/DAOD6000-0.pdf

3. Load the PageIndex JSON tree
   The script also loads the structured JSON tree created earlier by PageIndex.
   Example:

   results/DAOD6000-0_structure.json

4. Start local LLM connection
   The script connects to the local vLLM server.
   In your setup, this is:

   http://localhost:8010/v1

   The model you used is:

   Qwen/Qwen2.5-7B-Instruct-AWQ

5. Give the LLM tool instructions
   The LLM is told that it has access to three tools:

   get_document()
   get_document_structure()
   get_page_content(pages="5-7")

   These tools are Python functions inside your script.

6. Call get_document()
   The LLM first asks for basic document information.
   This returns metadata like:

   document name
   page count
   loaded status

7. Call get_document_structure()
   The LLM then asks for the PageIndex tree.
   This gives it the document map, for example:

Definitions pages 2-3
Policy Direction page 4
Authorities pages 5-7
References page 7
Other References page 8

8. Reason over the tree
The LLM looks at the user question and the PageIndex tree to decide which section is most relevant.

Example question:

What authority does ADM(IM) have?

The LLM sees that the likely relevant section is:

Authorities → pages 5-7

9. Call get_page_content()
The LLM then requests the actual page text using a page range.
Example:

```
{
  "pages": "5-7"
}
```

The script reads pages 5, 6, and 7 from the PDF.

10. Insert retrieved page text into the prompt
The retrieved PDF page text is passed back to the LLM as context.

This is the actual RAG retrieval step:

selected pages from PDF → LLM context

11. Generate final answer
The LLM answers the user's question using only the retrieved page text.

12. Print final answer
The script prints the final response in the terminal.

13. Optional verbose mode
If --verbose is enabled, the script shows each step:

raw model output
tool call
tool reason

tool output preview
final answer

The preview may look cut off because the script only prints the first part of the tool output.
The LLM still receives the full tool output.